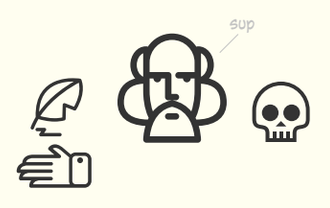
Developer Experience

HOW WRITERS ^PREFER TO WORK

a BEST PRACTICES mural in which we INFORM, DELIGHT and provide CLARITY ✨



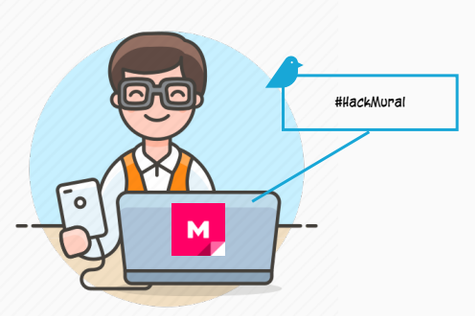
WHO

who we are

On the **Developer Experience** team, we inspire developers to

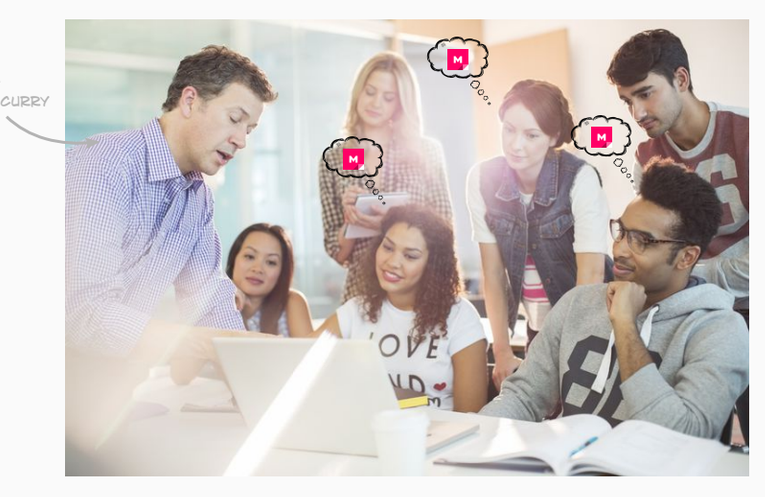
#HACK_MURAL

... to learn more about the product by creating add-ons, third-party workarounds and tools, and intentionally (gently) breaking it.



Because we are developer-focused, we work primarily with teams who handle API development and SDK development.

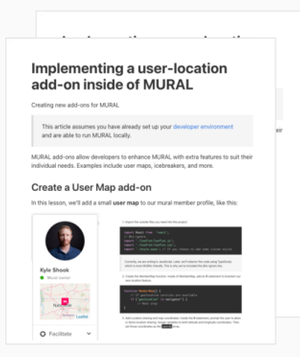
NOT TIM CURRY



Based on MURAL's API and SDK development, we brainstorm content to **inspire**, **inform** and attract developers.

blog

docs



Implementing a user-location add-on inside of MURAL

Welcome!

#HACK-MURAL
DEVELOPERS.MURAL.CO

WHAT

what we write

documentation.

What is documentation?

Documentation is ✎ content about a product or its pieces.

For the Developer Experience team, documentation is usually:

- **Guides**
  Multi-section docs about what a feature is & how it works
- **Use case articles**
  How to accomplish certain tasks with a feature
- **Other stuff**
  Such as in-product descriptions & support content

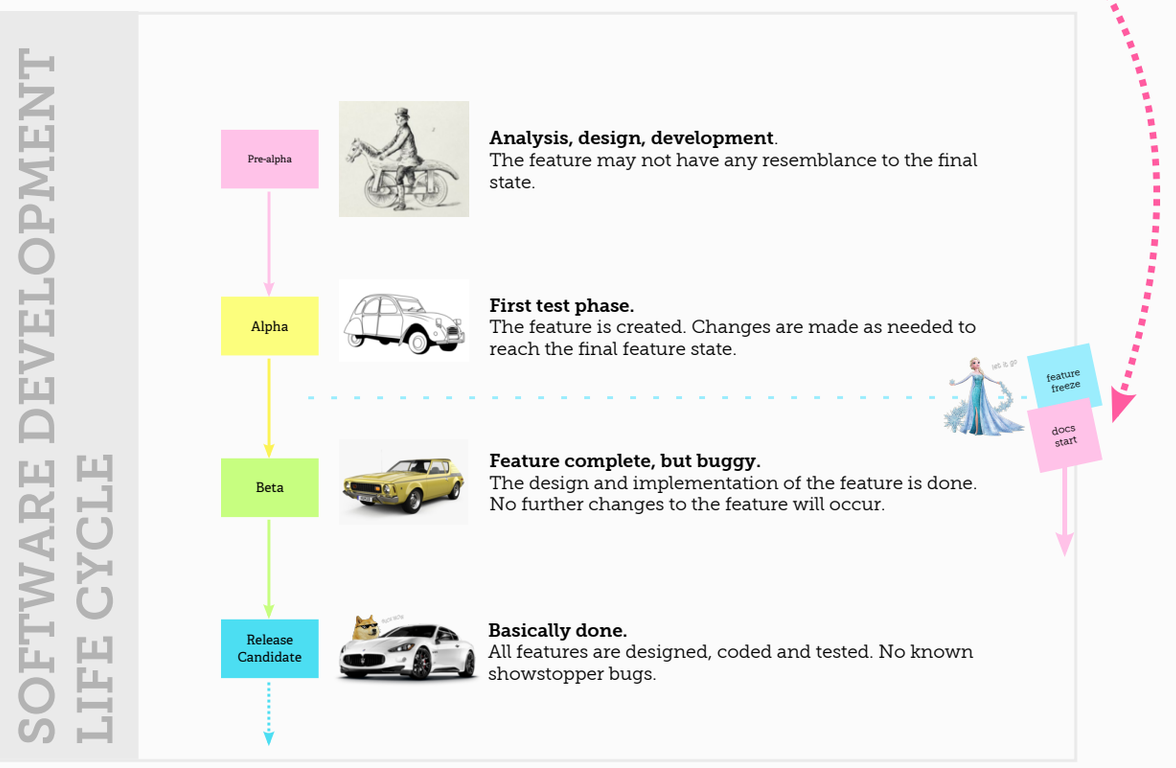
To keep it simple, we'll refer to the documented pieces of the product as **features**.

When does documentation begin?

In order to document a feature, the feature **must be mature enough** to show its function and place within the larger product set. It should look and act reasonably close to its intended release state.

If a feature is not complete, the writer may need to rewrite content multiple times, potentially **wasting resources**.

In order to avoid this duplication of work, **documentation cannot begin before feature freeze**. Feature freeze usually occurs as a product or feature goes from alpha to beta.



SOFTWARE DEVELOPMENT LIFE CYCLE

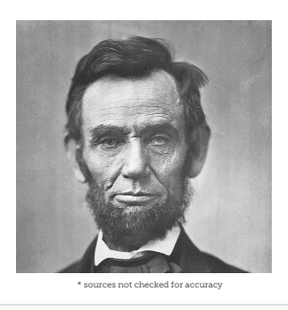
Pre-alpha — **Analysis, design, development.** The feature may not have any resemblance to the final state.

Alpha — **First test phase.** The feature is created. Changes are made as needed to reach the final feature state.

Beta — **Feature complete, but buggy.** The design and implementation of the feature is done. No further changes to the feature will occur.

Release Candidate — **Basically done.** All features are designed, coded and tested. No known showstopper bugs.

A note on timelines



Documentation takes time to create.

Good documentation, doubly so.

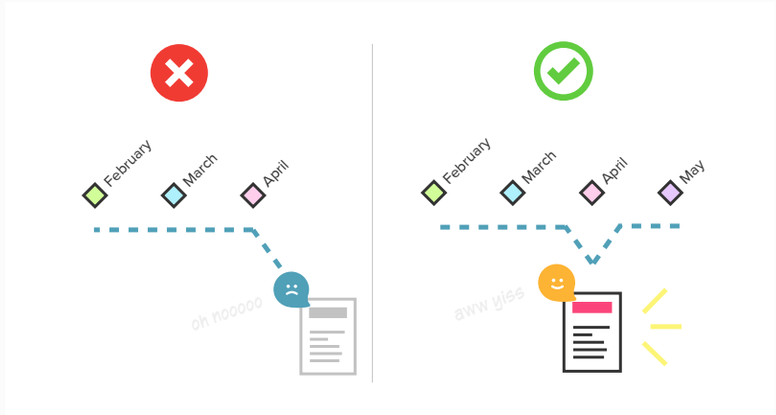
Abraham Lincoln*

* sources not checked for accuracy

Quite often, a team that rushes to finish a feature on time does not take **documentation time** into account.

Unfortunately, documentation can be treated as an afterthought (gasp!) or only started as a project nears release. This creates stress, rushed/incomplete documentation, and other problems.

{ Always include time for docs! }



blog.

What do we blog about?

Our **blog posts** are fun, inspiring articles about how to #Hack MURAL.

For example:

- Create a custom feature embedded in a mural.
- Use third-party tools to extract or inject custom data.
- Gamification of content, and other exciting developer stuff!

We're just getting started with our blog content. Stay tuned for great things!

How do we kick off blog posts?

A key part of our jobs is to **think like a developer**, so it's important for us to stay in touch with MURAL developers and engineers. This way, we can both keep up with exciting changes and be inspired by experiments into new tech.

Then, we gather our team for **brainstorming sessions** to come up with cool ideas for articles. We come away from these sessions with a list of articles that we'll need to create content for.

As a team, we **develop** applications and experiment, **learn** more about how the product works, and **write** the actual blog content.

Picture not available

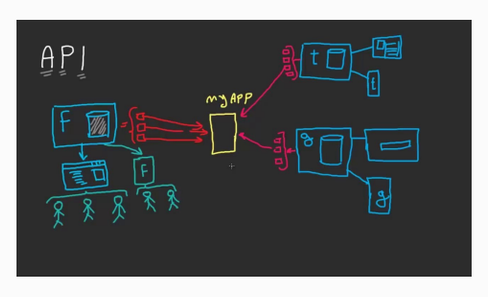
api spec.

What do we do with the API spec?

The API specification defines what the API does: How the endpoints interact with the MURAL product, and what the API user can expect for results.

Coders and engineers make the software work. Writers swoop in when needed and help write descriptive text or clarify the words describing a function.

We work as a team with the developers to make our API content clear and easy to follow.



HOW

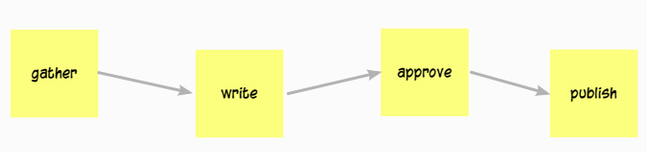
the writer's process

overview.

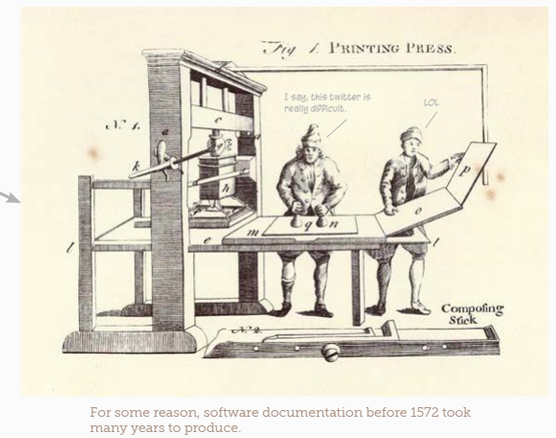
What does the writer do?

A writer's work can be summed up into four parts:

- **Gather**. Whether this is learning about a new feature to be documented or a brainstorming session with developers and engineers, the writer collects information and prepares to create.
- **Write**. The writer creates an outline for the project. Fills in the content. Identifies gaps. Engages their SME to answer questions and provide more detail.
- **Approve**. Once the content is drafted, it moves through a multi-stage approval process to be sure it is accurate and polished.
- **Publish**. The content is moved to the publishing platform. At the right time, it is made public, and the stakeholders—teams who care—are informed.



Once written and reviewed, the content is published!



For some reason, software documentation before 1572 took many years to produce.

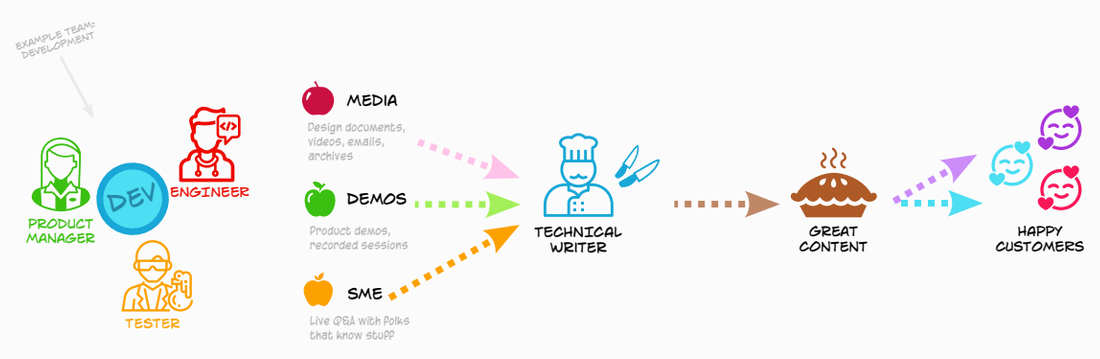
gather.

How is information gathered?

Once a feature is ready, the writer collects information from various sources. They use this information to craft **content**.

While the writer gets familiar with features over time, they are not experts. Writers work **with** experts to gather knowledge and teach others through content.

Other teams provide **media**, **demos**, and at least one **SME** to communicate with.


EXAMPLE TEAM: DEVELOPMENT
PRODUCT MANAGER
DEV
ENGINEER
TESTER
MEDIA
Design documents, videos, emails, archives
DEMOS
Product demos, recorded sessions
SME
Live Q&A with folks that know stuff
TECHNICAL WRITER
GREAT CONTENT
HAPPY CUSTOMERS

write.

What happens during the writing process?

What does "writing" actually entail?

Once we have enough material to begin, we ingest it, read it, watch it. We learn about the product and play with it as much as possible.

We create an outline. How does the content start? What do we cover? Where does it end? This outline is often reviewed and approved by the team to ensure that we're all on board with what is being covered and how.

We draft. The actual forming of words is done. Keys are punched. Our content is usually drafted in a review-friendly platform like Google Docs. During this drafting phase, the following happens:

- We write the parts we know.
- We ask the SME questions to clarify details and understand the product.
- We might use internal notes, placeholders, temporary images.

We peer-review. Throughout this process we have checkpoints with the team. We correct each other's work, make suggestions, and check that the story matches the outline.

approve.

How do approvals work?

The expanded approval process

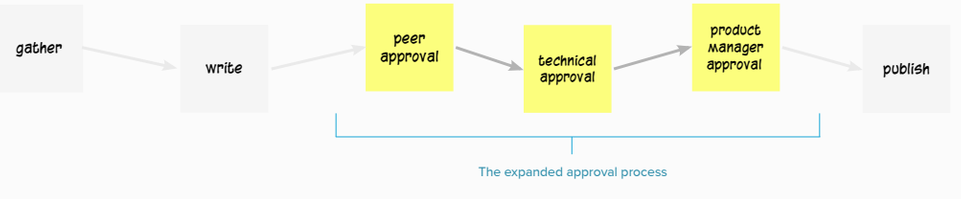
However, the approval process is a bit more complex. To ensure that our content is correct, well-written, and represents the product as best it can, **multiple reviewers** are engaged.

The content must move successfully through each phase of the approval process in order to be published. **If the content is rejected, it is updated or rewritten as needed.**

DO NOT PASS GO
REJECTED

At a minimum, content must be approved by a team member (peer) and two external reviewers.

Here are the steps:



A **peer approval** by another writer helps polish the wording and structure.

A **technical approval** by the SME makes sure we are on point with our technical details.

A **product manager approval** checks that the content meets our company standards, represents the product, and is what the customer needs.

publish.

What does publishing mean?

At long last: Publishing

Once all reviewers have approved the content, it is deemed ready for public consumption. Here are the steps:

We stage the content. Staging includes moving the final approved text and images to the publishing platform.

- Documentation is hosted on our *readme.io* sites.
- Our blog is hosted on *dev.to*.
- There may be other platforms, depending on where we're pushing content to.

Staging also includes reformatting and reviewing the content to be sure it will look perfect when we make it public.

We publish the content. At the approved time, we publish the content. This is essentially "flipping the switch" to make it publicly viewable. Sometimes, content is published as soon as it is staged. Other times, it is timed to coincide with a product release, or published on a recurring schedule, like our blog.

We notify stakeholders. We let teams who rely on this content know that it is published. This is usually a simple Slack message to the right channels, and a link to the final **product**.

Then ... we breathe. But just for a moment.

I love you, now go

the actual publishing process

fin.